New issue

unable to find Static system library 'gcc_eh' #17268

Closed #22167

Labels enhancement linking

Milestone 0.14.0

serjflint opened on Sep 25, 2023

Zig Version

0.11.0

Steps to Reproduce and Observed Behavior

Hi! I am trying to statically cross-compile Ruff from Linux to Linux-ARM (aarch64-unknown-linux-gnu). At the linking stage with command

```
TARGET=aarch64-unknown-linux-gnu
PKG_CONFIG_ALLOW_CROSS=1 cargo zigbuild --release --target=$TARGET --verbose
```

I get an error

```
error: linking with `/home/sandbox/.cache/cargo-zigbuild/0.17.3/zigcc-aarch64-unkno
  |
  = note: LC_ALL="C" PATH="/usr/local/rustup/toolchains/1.72-x86_64-unknown-linux-gnu/lib
  = note: error: unable to find Static system library 'gcc_eh' using strategy 'no_fallbac
            /place/rust/contrib/ruff/target/aarch64-unknown-linux-gnu/release/deps/libgcc
            /place/rust/contrib/ruff/target/release/deps/libgcc_eh.a
            /usr/local/rustup/toolchains/1.72-x86_64-unknown-linux-gnu/lib/rustlib/aarch6
            /usr/local/rustup/toolchains/1.72-x86_64-unknown-linux-gnu/lib/rustlib/aarch6
          error: unable to find Static system library 'gcc' using strategy 'no_fallback'.
            /place/rust/contrib/ruff/target/aarch64-unknown-linux-gnu/release/deps/libgcc
            /place/rust/contrib/ruff/target/release/deps/libgcc.a
            /usr/local/rustup/toolchains/1.72-x86_64-unknown-linux-gnu/lib/rustlib/aarch6
            /usr/local/rustup/toolchains/1.72-x86_64-unknown-linux-gnu/lib/rustlib/aarch6

error: could not compile `ruff_python_formatter` (bin "ruff_python_formatter") due to pre
```

```
Caused by:
  process didn't exit successfully: `/usr/local/rustup/toolchains/1.72-x86_64-unknown-lin
```

I have config.toml like this:

```
[target.aarch64-unknown-linux-gnu]
linker = "clang"
rustflags = [
  "-C", "target-feature=+crt-static",
]
```

The maintainer of cargo-zigbuild advised me to open an issue here
rust-cross/cargo-zigbuild#186

Expected Behavior

Successful build as for Linux-x86_64


  **serjflint** added bug on Sep 25, 2023

  **polarathene** mentioned this on Mar 7, 2024

 `cargo zigbuild` doesn't support static glibc builds with an explicit `--target` rust-cross/cargo-zigbuild#231

 **polarathene** on Mar 7, 2024 · edited by polarathene Edits

You can reproduce this with a basic hello world program in Rust. That should simplify tracking down the cause.

~~While I haven't tried with Zig directly for an equivalent Hello World program, at a glance it looks like Zig supports static builds just fine.~~ (***EDIT:*** *`zig build` vs `zig cc` is a different story it seems*)

~~It's more likely to be an incompatibility with how~~ ~~`cargo-zigbuild`~~ ~~is implemented?~~ ~~`libgcc`~~ ~~/~~ ~~`gcc_eh`~~ ~~are related to internals with the rust compiler or~~ ~~`cargo`~~ ~~IIRC. Perhaps there is some logic there that needs to be carried over to~~ ~~`cargo-zigbuild`~~ 

EDIT: I am partially wrong.

- While those files to link are related to the cargo build, Zig cannot compile a basic `hello.c` with static linking (*the advice there doesn't seem to apply to `libc.a` as it'd compile but segfault when run on the same build host*).
- Zig needs to fix their `cc` static build support before `cargo-zigbuild` can do anything about it.

polarathene mentioned this on Mar 8, 2024

zig cc: detect -static and treat -l options as static library names #4986

alexrp on Dec 6, 2024 Member

I'm pretty sure we can (and should) satisfy libgcc_eh by linking Zig's bundled libunwind, just as we do for libgcc_s.

alexrp added enhancement linking and removed bug on Dec 6, 2024

alexrp added this to the 0.14.0 milestone on Dec 6, 2024

alexrp added a commit that references this issue on Dec 6, 2024

`compiler: Classify libgcc_eh as an alias for libunwind.` ... Verified d0c962e

alexrp mentioned this on Dec 6, 2024

compiler: Classify various compiler-rt and libunwind names accurately and satisfy them #22167

alexrp added a commit that references this issue on Dec 6, 2024

`compiler: Classify libgcc_eh as an alias for libunwind.` ... Verified e7169e9

polarathene on Dec 7, 2024

@alexrp while that will resolve the linking issue, AFAIK `zig cc` will still enforce dynamic linking for `glibc` and static linking for `musl`.

Anyone that lands here when this issue is closed will want to still only use `musl` for static builds.

Normally you would want to be wary of `glibc` static linking of course as there are known caveats, but for Rust there is `eyra` which hooks into the `-gnu` / `glibc` target for providing it's own libc ABI implementation without the static glibc caveats.

However even with the ability to opt-out of dynamic linking, I recall another issue with Zig was that it also forcibly linked it's own CRT? `eyra` would provide it's own IIRC (*Building `eyra` without Zig still requires a linker flag `-nostartfiles`*).

alexrp on Dec 7, 2024 Member

Is this the only open issue we have tracking static glibc support? If so, the issue title is somewhat suboptimal...

polarathene mentioned this on Dec 7, 2024

RFC/Proposal: Turning Zig target triples into quadruples #20690

polarathene on Dec 7, 2024

> Is this the only open issue we have tracking static glibc support?

No, there's a few IIRC. At least this one for static glibc, and another for users that would like dynamic linking for musl (*such distros have their `cargo` package build modified for example to patch the musl target to dynamic link by default instead*).

andrewrk on Dec 7, 2024 Member

I don't think it makes sense to support "static glibc" as an explicit feature, and such a use case is not motivating to me in the slightest. However if an argument can be made that the use case is not working due to bugs or other surprising behavior, then perhaps the use case can be made to work.

Zig *is* supposed to support linking against the object files and other build artifacts installed on one's system, so if that includes a static glibc, then it should be possible to do even without Zig having explicit support for it.

polarathene on Dec 7, 2024

> I don't think it makes sense to support "static glibc" as an explicit feature, and such a use case is not motivating to me in the slightest.

I understand that, and don't really endorse static glibc either. However I think eyra is a valid use-case example which Zig is presently incompatible with.

Dynamic linking for musl is also a valid use-case, so having the ability to explicitly request static or dynamic linking would fix these build concerns.

> Zig *is* supposed to support linking against the object files and other build artifacts installed on one's system, so if that includes a static glibc, then it should be possible to do even without Zig having explicit support for it.

I believe I tried to do that here, but due to the enforced dynamic linking flag, even though there are no libraries dynamically linked on the executable, it segfaults. I'd consider that a bug?

The other issue with compatibility with eyra also involves opt-out via linker flag `-nostartfiles`. Rust would provide those by default otherwise and Zig appears to attempt the same AFAIK (*but additionally ignores this linker flag*). I don't think there is an existing issue regarding this flag, but there are a few issues (*all related to `zig cc` with Rust*), this one in particular seems to be relevant (*focused on musl with later comments suggesting workarounds that would not work for eyra*).

andrewrk on Dec 7, 2024 · edited by andrewrk Edits Member

> I believe I tried to do that here, but due to the enforced dynamic linking flag, even though there are no libraries dynamically linked on the executable, it segfaults. I'd consider that a bug?

Yeah sure, I buy that argument.

My above comment is mainly in response to **@alexrp**.

alexrp on Dec 7, 2024 Member

Just to be clear, I don't personally care about static glibc either, except to the extent that we're handling some flags wrong or whatever. I was just trying to figure out whether it makes sense to use this particular issue to track that, or if it makes more sense to close it with #22167 (which is what I was intending to do).

polarathene on Dec 7, 2024

This should be closed for resolving the `gcc_eh` issue. My earlier comment in response to the PR was for other users and the issue author to be aware that static linking glibc will still fail.

#4986 is probably sufficient for tracking static glibc (or equivalent fix) support?

andrewrk closed this as completed in #22167 on Dec 7, 2024

Add a comment

Write | Preview

Use Markdown to format your comment

Paste, drop, or click to add files

Reopen Issue | Comment

Remember, contributions to this repository should follow its contributing guidelines and code of conduct.

Metadata

Assignees

No one assigned

Labels

enhancement linking

Type

No type

Projects

No projects

Milestone

0.14.0

Closed 3 weeks ago, 100% complete

Relationships

None yet

Development

compiler: Classify various compiler-rt and libunwind names accurately and satisfy them

ziglang/zig

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

Participants